

Running Notes-30. LangChain & Agentic AI – From Basics to Advanced 30

Stateless LLM Calls & Manual Conversation History

1. Program 1 — Prove That LLM Calls Are Stateless

Two separate `invoke()` calls do **not automatically share previous conversation**.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
response1 = llm.invoke(  
    "My name is Durga."  
)
```

```
print("FIRST RESPONSE")
```

```
print("-----")
print(response1.content)
```

```
response2 = llm.invoke(
    """
    What is my name?

    If my name is not available in the
    supplied conversation, say "I don't know".
    """
)
```

```
print("\nSECOND RESPONSE")
print("-----")
print(response2.content)
```

2. Understanding the Two Calls

Call 1

"My name is Durga."



Model



AI Response

Call 2

"What is my name?"



Model



AI Response

There is **no automatic connection** between these two independent calls.

3. What Is Statelessness?

Stateless means an independent model call does not automatically contain the previous application calls.

Important Notes

1. The model cannot use information from an earlier independent call unless our application supplies it again.
2. Using the same `llm` object does **not** mean the same conversation continues automatically.

4. Same LLM Object Does Not Mean Same Conversation

Think of the `llm` object like a phone.

You may use the same phone to make many calls.

But every call does not automatically contain all previous conversations.

Similarly:

`llm.invoke(...)`

can be called many times.

But each independent call only gets the input we provide to it.

5. Then Why Does ChatGPT Look Like It Remembers?

A chat application can maintain conversation history and provide relevant previous messages together with the new message.

Concept

APPLICATION MAINTAINS HISTORY



MODEL USES THE HISTORY

The model can respond using the supplied history.

6. What Is Conversation History?

Conversation history is the sequence of previous messages exchanged between the user and the AI.

Example:

HumanMessage



AIMessage



HumanMessage



AIMessage

7. Program 2 — Manually Supply Conversation History

Here we manually provide the previous messages to the model.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import (
    HumanMessage,
    AIMessage
)

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

messages = [
    HumanMessage(
        content="My name is Durga."
    ),

    AIMessage(
        content="Nice to meet you, Durga."
    ),

    HumanMessage(
```

```
        content="What is my name?"
    )
]
```

```
response = llm.invoke(messages)
```

```
print(response.content)
```

8. Why Can the Model Answer Now?

Because the model receives:

"My name is Durga."

+

"Nice to meet you, Durga."

+

"What is my name?"

The required information is available inside the supplied conversation.

9. Program 3 — Without History vs With History

This is one of the most important demonstrations.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import (
```

```
    HumanMessage,
```

```
    AIMessage
```

```
)
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
question = HumanMessage(
```

```
    content="""
```

```
    What is my name?
```

```
    If my name is not available in the
```

supplied conversation, say "I don't know".

"""

)

WITHOUT HISTORY

response1 = llm.invoke(

[question]

)

print("WITHOUT HISTORY")

print("-----")

print(response1.content)

WITH HISTORY

messages = [

HumanMessage(

```

        content="My name is Durga."
    ),

    AIMessage(
        content="Nice to meet you."
    ),

    question
]

response2 = llm.invoke(
    messages
)

print("\nWITH HISTORY")
print("-----")
print(response2.content)

```

10. Without History

Current Question



Model



Response

The model receives only:

"What is my name?"

So the required information may not be available.

11. With History

Previous Human Messages

+

Previous AI Messages

+

Current Human Message



Model



Context-Aware Response

Now the model has the required conversation context.

12. Important Observation

Without History

Question



Model



No previous context

With History

Previous Messages

+

Current Question



Model



Context-Aware Answer

13. Conversation History Gives Memory-Like Behaviour

From the user's point of view, it may look like:

"AI remembers me"

But technically, at this stage:

Application stores history



Application sends history



Model uses supplied history

14. Conversation History Does Not Mean Model Training

Very important.

Providing previous messages does **not** mean we are retraining the model.

Conversation History

Previous messages



Sent as context



Model responds

Model Training

This is a different concept.

So:

Conversation History

≠

Model Training

15. Program 4 — Favourite Technology Example

This example shows that the model can answer because the required information exists in the supplied history.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import (
```

```
    HumanMessage,
```

```
    AIMessage
```

```
)
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```

    api_key=os.getenv("OPENAI_API_KEY")
)

messages = [
    HumanMessage(
        content="My favorite technology is Python."
    ),

    AIMessage(
        content="Great choice."
    ),

    HumanMessage(
        content="What is my favorite technology?"
    )
]

response = llm.invoke(messages)

print(response.content)

```

16. How This Works

The model receives:

"My favorite technology is Python."



"Great choice."



"What is my favorite technology?"

Therefore, it can use the supplied conversation information.

17. Program 5 — Remove History and Observe the Difference

Now remove the earlier message.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
messages = [  
    HumanMessage(  
        content="""  
        What is my favorite technology?  
  
        If this information is not available  
        in the supplied conversation,  
        say "I don't know".  
        """"  
    )  
]  
  
response = llm.invoke(messages)  
  
print(response.content)
```

18. Why Is the Answer Different?

The model does not receive:

My favorite technology is Python.

Therefore, it cannot use that information.

Main Rule

Information supplied



Model can use it

Information not supplied



Model cannot use it from that earlier independent call

19. Program 6 — Reuse the Actual AIMessage Returned by the Model

The response returned by a chat model is already an AIMessage.

So we can directly include it in the conversation history.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
first_message = HumanMessage(  
    content="My name is Durga."  
)
```

```
response1 = llm.invoke(  
    [first_message]  
)
```

```
print("FIRST RESPONSE")  
print("-----")  
print(response1.content)
```

```
print("\nTYPE")  
print("----")
```

```

print(type(response1))

messages = [
    first_message,

    # response1 is already an AIMessage
    response1,

    HumanMessage(
        content="What is my name?"
    )
]

response2 = llm.invoke(
    messages
)

print("\nSECOND RESPONSE")
print("-----")
print(response2.content)

```

20. Important Point

We do not always have to manually create:

`AIMessage(...)`

for the model response.

The response returned by the chat model can itself be reused.

Concept:

HumanMessage



Model



AIMessage returned



Reuse in history

21. First Connection to Application State

State is information an application needs to keep while working.

Conversation history is one example of state.

Example:

[

```
HumanMessage(...),  
AIMessage(...),  
HumanMessage(...),  
AIMessage(...)  
]
```

As the conversation continues:

State changes

For now, just remember:

Conversation History



One type of



Application State

22. Program 7 — Day 1 Master Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
# =====
```

```
# CREATE CHAT MODEL
```

```
# =====
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# =====
```

```
# STEP 1 — FIRST USER MESSAGE
```

```
# =====
```

```
first_message = HumanMessage(  
    content="My name is Durga."  
)
```

```
response1 = llm.invoke(  
    [first_message]
```

```
)
```

```
print("FIRST RESPONSE")
```

```
print("-----")
```

```
print(response1.content)
```

```
# =====
```

```
# STEP 2 — NEW CALL WITHOUT HISTORY
```

```
# =====
```

```
question = HumanMessage(
```

```
    content="""
```

```
    What is my name?
```

```
    If my name is not available in the
```

```
    supplied conversation,
```

```
    say "I don't know".
```

```
    """)
```

```
)
```

```
response2 = llm.invoke(  
    [question]  
)
```

```
print("\nWITHOUT HISTORY")  
print("-----")  
print(response2.content)
```

```
# =====  
# STEP 3 — SAME QUESTION WITH HISTORY  
# =====
```

```
messages = [  
    first_message,  
    response1,  
    question  
]
```

```
response3 = llm.invoke(  
    messages  
)  
  
print("\nWITH HISTORY")  
print("-----")  
print(response3.content)
```

23. Understanding the Master Program

Step 1

"My name is Durga."



Model



AI Response

Step 2 — Without History

"What is my name?"



Model



No earlier conversation supplied

Step 3 — With History

First Human Message

+

First AI Response

+

Current Question



Model



Context-Aware Response

24. Most Important Observation

Same llm object

Same question

BUT

Different input



Different available context



Different response

This clearly shows that the difference is not simply the llm object.

The important difference is:

What input/context we provide to the model

25. Same LLM Object — Different Context

Consider:

```
llm = ChatOpenAI(...)
```

We use the same object for both calls.

But:

Call 1

```
llm.invoke(  
    [question]  
)
```

Input contains only the question.

Call 2

```
llm.invoke(  
    [  
        first_message,  
        response1,  
        question  
    ]  
)
```

Input contains conversation history.

Therefore:

Same Model

+

Different Context

↓

Different Response

26. Program 8 — Classroom Exercise: City Example

Students can practise the same concept using city information.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import (
```

```
    HumanMessage,
```

```
    AIMessage
```

```
)
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
# WITHOUT HISTORY
```

```
response1 = llm.invoke(  
    [  
        HumanMessage(  
            content="""  
            Where do I live?  
  
            If this information is not available  
            in the supplied conversation,  
            say "I don't know".  
            """  
        )  
    ]  
)  
  
print("WITHOUT HISTORY")  
print("-----")  
print(response1.content)
```

```
# WITH HISTORY
```

```
messages = [  
    HumanMessage(  
        content="I live in Hyderabad."  
    ),  
  
    AIMessage(  
        content="Okay."  
    ),  
  
    HumanMessage(  
        content="Where do I live?"  
    )  
]
```

```
response2 = llm.invoke(messages)
```

```
print("\nWITH HISTORY")
```

```
print("-----")  
print(response2.content)
```

27. City Example Flow

Without History

"Where do I live?"

↓

Model

↓

Required information unavailable

With History

"I live in Hyderabad."

+

"Okay."

+

"Where do I live?"

↓

Model

↓

Can use supplied history

28. Common Mistakes

Mistake 1

Same llm object means same conversation.

Wrong.

The same llm object can be used for many independent calls.

Mistake 2

Every invoke() automatically receives previous invoke() calls.

Wrong.

Each independent call receives only the input supplied to that call.

Mistake 3

Conversation history means model training.

Wrong.

Conversation history is supplied as context.

It is not the same as model training.

Mistake 4

Only user messages are conversation history.

Wrong.

Conversation history normally contains both:

HumanMessage

+

AIMessage

Mistake 5

Creating a Python message list means the information is permanently stored forever.

Wrong.

At this stage, it is simply information maintained by the running application.

Persistence can be handled separately later.

29. Five Key Points

1. Separate model calls do not automatically contain previous application calls.
2. Conversation history is the sequence of previous Human and AI messages.

3. The application maintains and supplies conversation history.
4. The model uses the supplied history as context.
5. Conversation history is not the same as model training or permanent memory.

30. MCQ 1

Consider:

```
llm.invoke(  
    "My city is Hyderabad."  
)
```

```
llm.invoke(  
    "What is my city?"  
)
```

Does the second independent call automatically receive:

"My city is Hyderabad"

- A. Yes
- B. No

Answer: B

31. MCQ 2

Who is mainly responsible for maintaining conversation history in our application?

- A. StrOutputParser
- B. Pydantic
- C. Application
- D. Temperature

Answer: C

32. MCQ 3

Why can this model answer correctly?

```
messages = [  
    HumanMessage(  
        content="My city is Hyderabad."  
    ),  
  
    AIMessage(  
        content="Okay."  
    ),  
  
    HumanMessage(  
        content="What is the capital of India?"  
    )  
]
```

```
        content="What is my city?"
    )
]
```

- A. Model was retrained
- B. Required information exists in the supplied conversation
- C. HumanMessage stores information permanently
- D. invoke() automatically remembers every previous call

Answer: B

33. Stateless vs Conversation History

Stateless Call	With Conversation History
Previous call is not automatically included	Previous messages are supplied
Current input only	Previous + current messages
No automatic conversation continuity	Application provides continuity
Independent call	Context-aware call

34. Simple Architecture

Stateless Calls

Call 1

"My name is Durga"



Model



Response

Call 2

"What is my name?"



Model

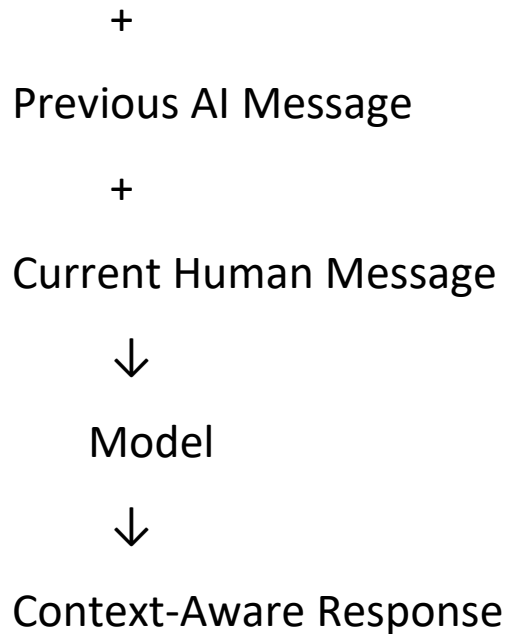


Response

No automatic link.

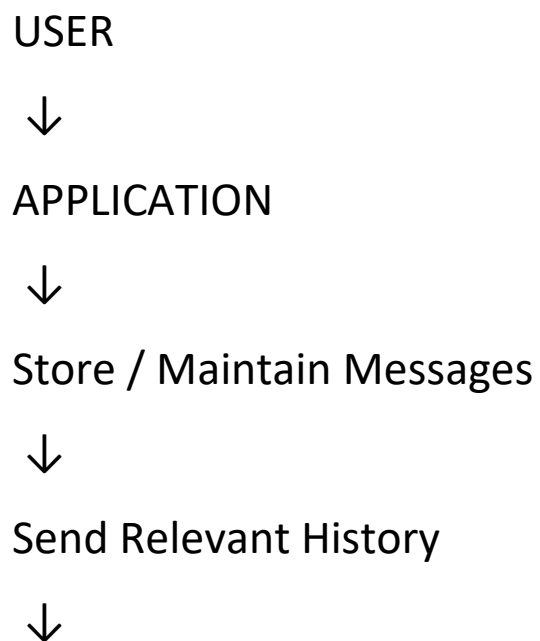
With Conversation History

Previous Human Message



35. Application Responsibility

The application is responsible for maintaining the required conversation data.



MODEL



RESPONSE

36. Important Memory Formula

MODEL CALL



STATELESS BY DEFAULT

To get conversation continuity:

APPLICATION HISTORY



SUPPLY HISTORY TO MODEL



CONTEXT-AWARE RESPONSE

37. HumanMessage and AIMessage

Conversation history usually contains:

HumanMessage(...)

for user messages.

and:

AIMessage(...)

for AI responses.

Example:

```
messages = [  
    HumanMessage(  
        content="My name is Durga."  
    ),  
  
    AIMessage(  
        content="Nice to meet you."  
    ),  
  
    HumanMessage(  
        content="What is my name?"  
    )  
]
```

38. Master Concept

LLM

|

- | Does not automatically carry
- | previous independent calls



STATELESS CALL

But the application can do:

Conversation History



Current Message



Send Together



Model



Context-Aware Response

39. Final Revision Points

1. LLM Calls Are Stateless

Independent calls do not automatically include previous calls.

2. Same LLM Object Does Not Mean Same Conversation

llm.invoke(...)

llm.invoke(...)

does not automatically create conversation continuity.

3. Conversation History

Conversation history contains previous messages exchanged between:

Human



AI

4. Application Maintains History

Application



Maintains Messages



Supplies Messages



Model

5. Model Uses Supplied Context

The model can use information only when that information is available in the supplied input/context.

6. History Can Contain Both Sides

HumanMessage

+

AIMessage

7. Returned Model Response Can Be Reused

The model response itself can be included in later history.

8. Conversation History Gives Memory-Like Behaviour

But technically:

Application maintains context

9. Conversation History Is Not Training

Conversation History

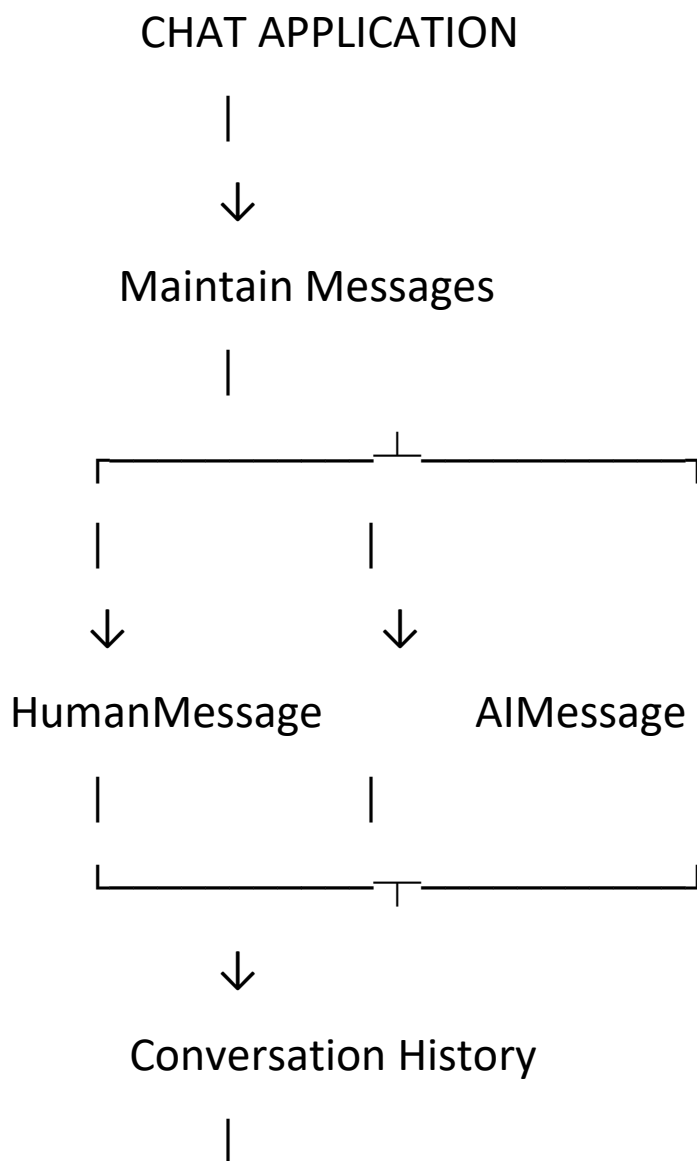
≠

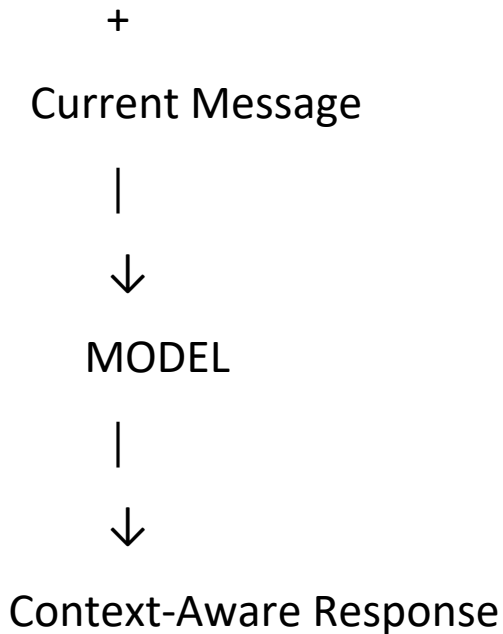
Model Training

10. Conversation History Is Application State

Conversation history is one example of information the application keeps while working.

40. Master Diagram





41. One-Line Day 1 Conclusion

The model does not automatically carry previous application calls; our application must maintain and provide the required conversation history.

42. Easy Memory Line

NO HISTORY



NO PREVIOUS CONTEXT

WITH HISTORY



PREVIOUS CONTEXT AVAILABLE

And remember:

Same LLM Object

≠

Same Conversation